Introduction

Basic
Combining labels
Combining outputs
Model Based

Bagging

Random Forests

Boosting
AdaBoost
Gradient boosting

Machine Learning: Ensembles

Machine Learning I - MCD

J. Muñoz and V. Laparra

ISP - ETSE - Universitat de València

VLC - 2024

Outline

1 Introduction

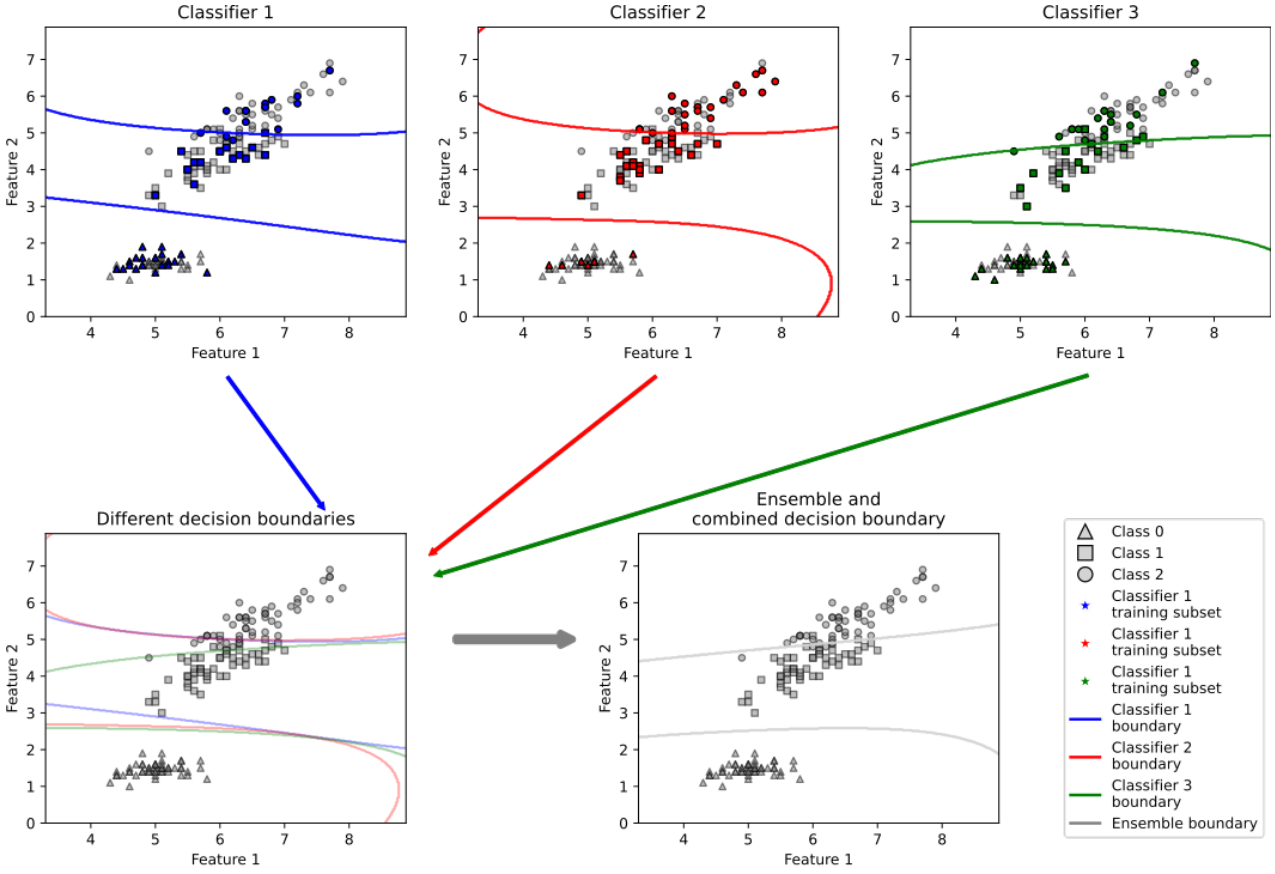
2 Basic
- Combining labels
- Combining outputs
- Model Based

3 Bagging

4 Random Forests

5 Boosting
- AdaBoost
- Gradient boosting

Introduction



It is used continously in AI!

Introduction

Reasons for use ensembles:

- Statistical improvements: the combination may not be better than one of the individual predictors, but it reduces the overall risk of making a poor prediction.
- Large quantity of data: it is not feasible to train models with large amount of data, but it is possible to partition the data into smaller subsets.
- Low quantity of data: ensemble methods based on sampling with repetition can solve the problem of having small datasets.
- Divide and conquer: some problems are too difficult to be solved for just one classifier. The classifier would be too complex. On the other hand, combining simple (even linear) classifiers could solve difficult problems efficiently.
- Data fusion: samples coming from different sources are difficult to use in a single predictor. It is easier and convenient to use different predictors for different data sources.

Combining labels I

Notation: We have T classifiers, $t = 1, \ldots, T$, C classes, $j = 1, \ldots, C$. The output of each classifier is $d_{t,j} \in \{0, 1\}$.

- **Majority voting**
  - Choose class ω_J if

$$\sum_{t=1}^{T} d_{t,J} = \max_{j=1}^{C} \sum_{t=1}^{T} d_{t,j}$$

- **Weighted majority voting**
  - Similar to majority voting, useful when we know a priori that some predictors are better than others
  - Assign a weight w_t to each predictor, then choose class ω_J if

$$\sum_{t=1}^{T} w_t d_{t,J} = \max_{j=1}^{C} \sum_{t=1}^{T} w_t d_{t,j}$$

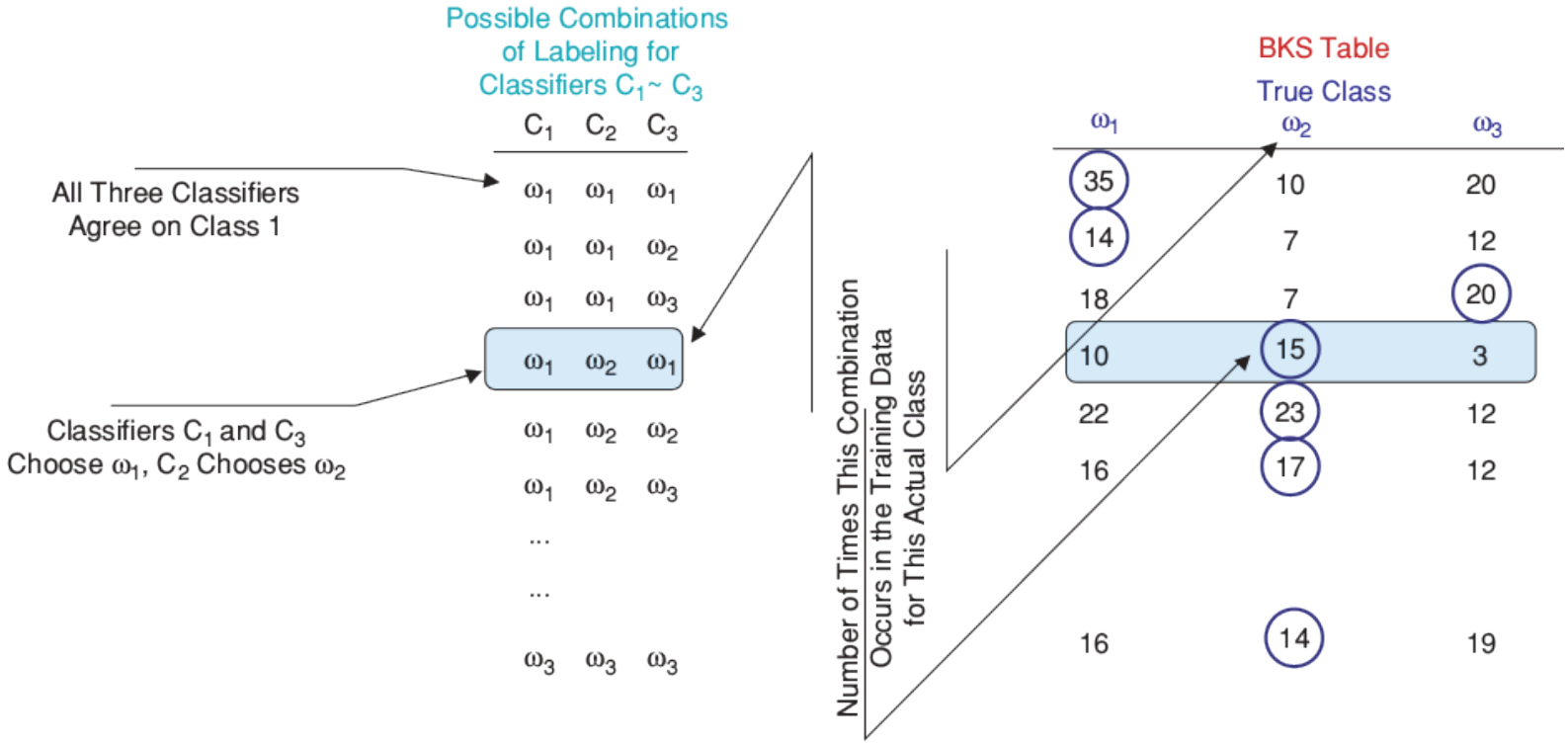
  - By construction, AdaBoost follows this strategy

Combining labels II

- **Behaviour Knowledge Space (BKS)**
  - Proposed by Huang and Suen[1]
  - When training, it builds a look up table that tracks how often each labelling combination is produced by the classifiers.
  - In test, given the predicted combination, it assigns the most observed true class for that combination (see example in next slide).
  - Example: ensemble prediction $\omega_1\omega_2\omega_1$ will get ω_2 as class (as opposed of ω_1 given by majority voting).
  - Trained with sufficient training data, and normalizing the table, the class obtained by BKS is a good estimation of the class posterior probability.

[1] Y.S. Huang and C.Y. Suen, "A method of combining multiple experts for the recognition of unconstrained handwritten numerals," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 17, no. 1, pp. 90–94, 1995

BKS look up table example



Combining outputs I

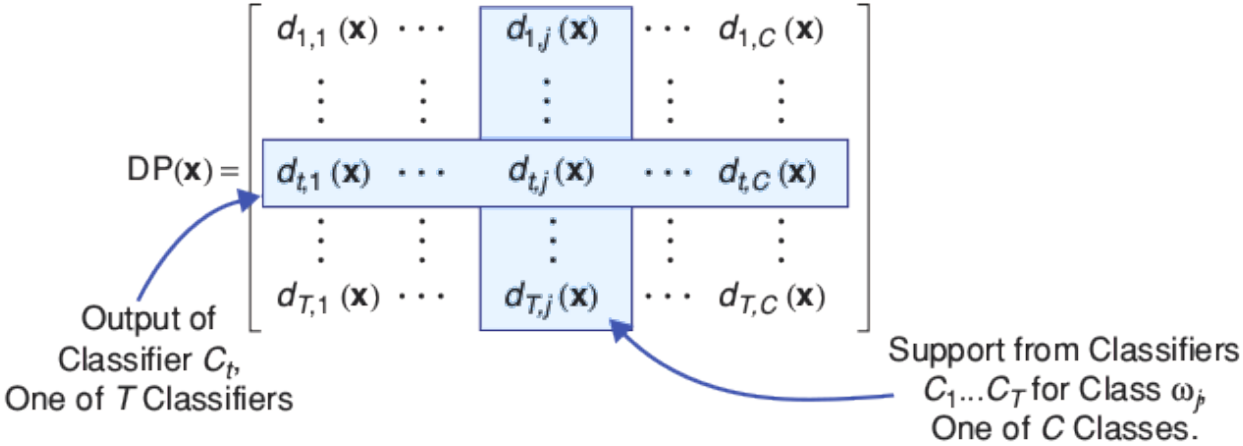
- These methods are valid for classifiers that give a continuous output.
- This c.o. is the degree of support given by a classifier to a class.
- It is usually considered an estimate of the posterior probability of that class using the *softmax* normalization[2]:

$$P(\omega_j|\boldsymbol{x}) \approx g_j^*(\boldsymbol{x}) = \frac{e^{g_j(\boldsymbol{x})}}{\sum_{c=1}^{C} e^{g_c(\boldsymbol{x})}} \Rightarrow \sum_{c=1}^{C} g_j^*(\boldsymbol{x}) = 1$$

where $g_j(\boldsymbol{x})$ is the classifier output for the j class.

Combining outputs II

- Kuncheva *et al.*[3] introduced the decision profile matrix, $D(\boldsymbol{x})$
- Rows: classifiers
- Columns: clases



[2] R.O. Duda, P.E. Hart, and D. Stork, "Algorithm Independent Techniques," in Pattern Classification, 2 ed New York: Wiley, pp. 453–516, 2001

[3] L.I. Kuncheva, J.C. Bezdek, and R. Duin, "Decision templates for multiple classifier fusion: An experimental comparison," Pattern Recognition, vol. 34, no. 2, pp. 299–314, 2001

Borda

Borda Count

- Introduced by Jean Charles de Borda in 1770 !!
- Compared to other combination methods, Borda Count does not throw away the support given to a non-winning class
- The classifier must output a rank ordered classification (easily obtained in those providing continuous outputs, i.e. post. prob. estimates)
- With J classes (candidates), first one obtains $J-1$ votes, second, $J-2$ votes, and so on. Last candidate receives 0 votes.
- Votes are added up across all classifiers. The class with more votes wins.

Curiosities :-) Used in ...

- Selecting the most valuable player in US baseball league
- Ranking university in college sports
- A similar system is used to choose the winning song in Eurovision

Algebraic combiners (aggregation) I

Mean rule: the support for class ω_j is given by the average sum of all classifiers' jth outputs

$$\mu_j(\boldsymbol{x}) = \frac{1}{T}\sum_{t=1}^{T} d_{t,j}(\boldsymbol{x})$$

The selected class if the one obtaining the largest $\mu_j(\boldsymbol{x})$.

Algebraic combiners (aggregation) II

Weighted average: similar to weighted majority voting, but applying weights to continuous outputs. Two options:

1. A weight per classifier

$$\mu_j(\boldsymbol{x}) = \frac{1}{T}\sum_{t=1}^{T} w_t d_{t,j}(\boldsymbol{x})$$

2. A weight per classifier and per class

$$\mu_j(\boldsymbol{x}) = \frac{1}{T}\sum_{t=1}^{T} w_{t,j} d_{t,j}(\boldsymbol{x})$$

Algebraic combiners (aggregation) III

Trimmed mean:

- Avoid very optimistic/pessimistic outputs. This may happens if one of more classifiers in the ensemble unusually outputs very high/low values.
- All classifiers' outputs are obtained, sorted, and a percentage of them are just removed from both ends.

Minimum, maximum, median:

$$\mu_j(\boldsymbol{x}) = \max_{t=1,\ldots,T}\{d_{t,j}(\boldsymbol{x})\}$$

$$\mu_j(\boldsymbol{x}) = \min_{t=1,\ldots,T}\{d_{t,j}(\boldsymbol{x})\}$$

$$\mu_j(\boldsymbol{x}) = \operatorname{median}_{t=1,\ldots,T}\{d_{t,j}(\boldsymbol{x})\}$$

- The selected class is the one obtaining the *largest* $\mu_j(\boldsymbol{x})$ value.
- The median rule is eq. to trimmed rule at 50%.

Algebraic combiners (aggregation) IV

Product rule:

- Just as its name implies, predictors' outputs are multiplied.
- Very sensitive to pessimistic predictors. In any output is close to zero, that class won't be selected.
- If individual posteriors are well estimated, this rule provides the best estimate of the posterior probability given by the ensemble.

$$\mu_j(\boldsymbol{x}) = \frac{1}{T} \prod_{t=1}^{T} d_{t,j}(\boldsymbol{x})$$

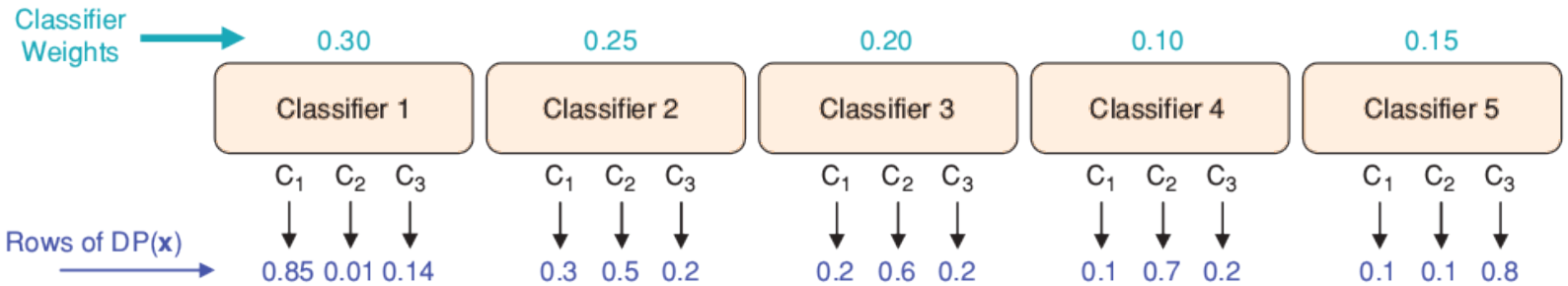
Algebraic combiners (aggregation) V

Generalized mean: The interest of this rule is that it contains some of the previous rules as particular cases:

$$\mu_j(\boldsymbol{x}, \alpha) = \left(\frac{1}{T} \sum_{t=1}^{T} d_{t,j}(\boldsymbol{x})^{\alpha} \right)^{1/\alpha}$$

- $\alpha \to -\infty$: minimum rule
- $\alpha \to \infty$: maximum rule
- $\alpha \to 1$: mean rule
- $\alpha \to 0$: $\mu_j(\boldsymbol{x}) = \left(\prod_{t=1}^{T} d_{t,j}(\boldsymbol{x}) \right)^{1/T}$ (modified version of product rule)

Example of combiners



Product Rule:	**C_1: 0.0179,**	C_2: 0.00021,	C_3: 0.0009
Sum Rule:	C_1: 1.55,	**C_2: 1.91,**	C_3: 1.54
Mean Rule:	C_1: 0.310,	**C_2: 0.382,**	C_3: 0.308
Max Rule:	**C_1: 0.85,**	C_2: 0.7,	C_3: 0.8
Median Rule:	C_1: 0.2,	**C_2: 0.5,**	C_3: 0.2
Minimum Rule:	C_1: 0.1,	C_2: 0.01,	**C_3: 0.14**
Weighted Average	**C_1: 0.395,**	C_2: 0.333,	C_3: 0.272
Majority Voting:	C_1: 1,	**C_2: 3,**	C_3: 1 Vote
Weighted Majority Voting:	C_1: 0.3,	**C_2: 0.55,**	C_3: 0.15 Votes
Borda Count:	C_1: 5	**C_2: 6,**	C_3: 4 Votes

Which one should we use?

- Several studies conclude that the *no-free-lunch* theorem also holds for combination rules.
- In this scenario, mean rule is often used due to its simplicity and consistent performance in many different applications.
- If the accuracies (importance) of the predictors can be reliably estimated, then use weighted rules instead.
- If the predictors' outputs are a good estimation of the posteriors, then the product rule should be used.

Exercise: Combinations (Aggregations)

Implement

1. Implement and compare 3 aggregation methods.

Stacked generalization

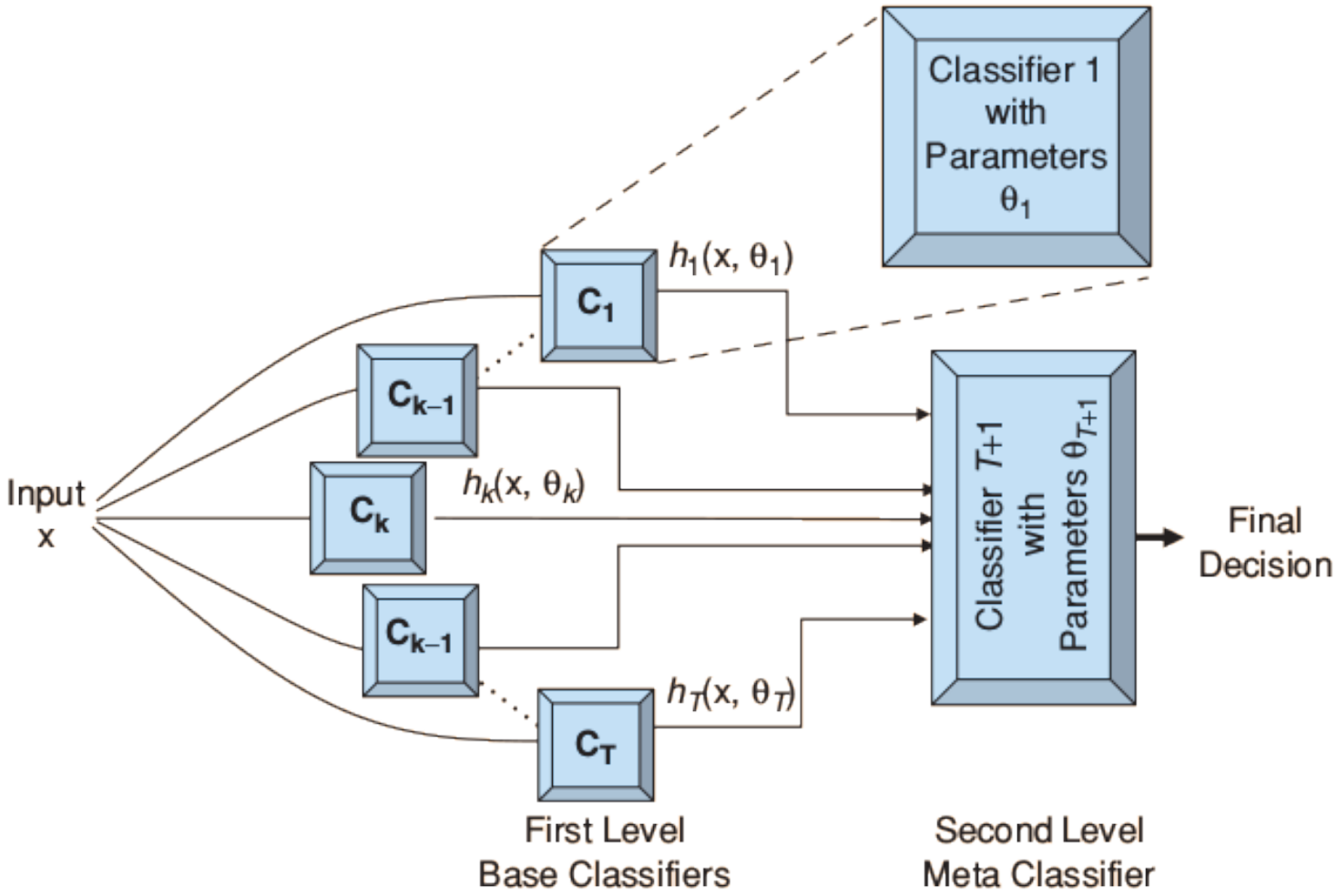
- Wolpert in 1992[4].
- Deduces the biases of an ensemble of predictors given a training set.
- Train a predictor model that maps the ensemble outputs to the correct answers.
- Goal: improving generalization accuracy (not learning accuracy)

Structure

1. A first level made of an ensemble of classifiers.
2. A second level with a meta-classifier that learns the mapping between their outputs and the right prediction.

[4] D.H. Wolpert, "Stacked generalization," Neural Networks, vol. 5, no. 2, pp. 241–259, 1992.

Stacked generalization schema



Stacked generalization

Training

1. Using k-fold divide training set into T blocks.
2. Train **all** $C_1, \ldots, C_T$ using the sets of $T-1$ blocks.
3. Obtain the predictions of **all** $C_1, \ldots, C_T$ on the unseen T blocks.
4. Train C_{T+1} using the $C_1, \ldots, C_T$ predictions and the true labels.
5. Retrain $C_1, \ldots, C_T$ on the entire training set.

Exercise: Stacked generalization

Implement

1. Implement Stacked generalization.
2. Use 3 or 4 level 1 classifiers of any type
3. Use `LogisticRegression` as level 2 classifier

Mixture-of-Experts[5]

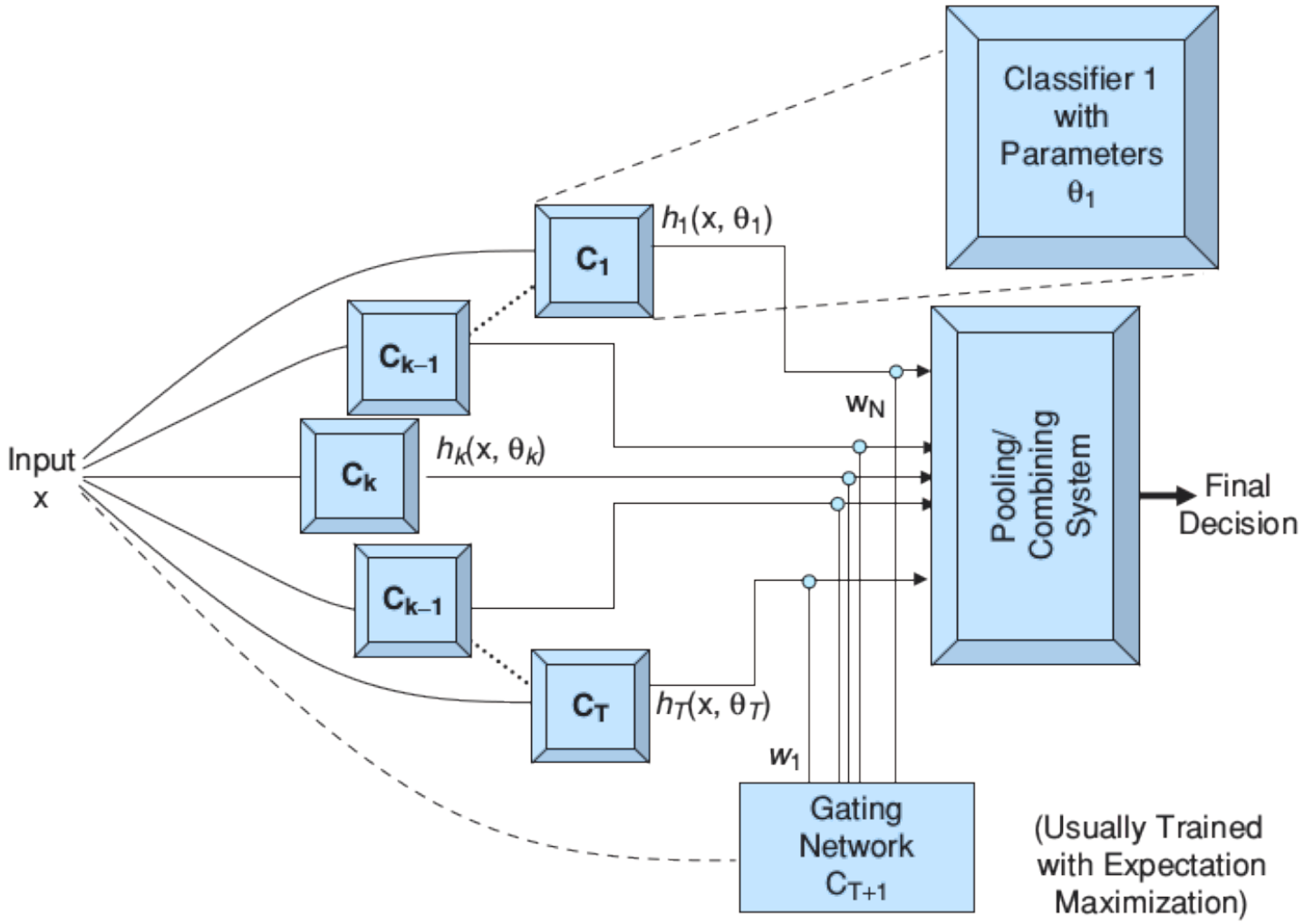
- Similar to stacked generalization, but level 2 classifier just assigns the weights of a combiner.
- Weights are adapted by sample!

The key is the level 2 classifier, called *gating network*

- Determines the weight distribution per sample
- A NN trained with backpropagation or EM
- Inputs are the training samples
- We have a dynamic combination rule!

[5] R.A. Jacobs, M.I. Jordan, S.J. Nowlan, and G.E. Hinton, "Adaptive mixtures of local experts," Neural Computation, vol. 3, pp. 79–87, 1991

Mixture-of-Experts Structure



Bagging by weak ensembles

Introduced by Leo Breiman in 1994.

- Bagging = bootstrap + aggregation.
- Build training sets by sampling training data with repetition (bootstrap).
- Use these training sets to train weak predictors[6].
- Combine the output of w.p. to obtain a complex predictor.
- For regression combine them using the mean.
- For classification, majority voting.
- Good candidates for w.p. are Trees, Naive Bayes, shallow NNs.

The advantage is that weak learners are trained faster than complex algorithms. They present also high bias and variance, something that gets fixed when using many of them.

[6] A predictor which performs just above the chance baseline

Bagging: formally I

- Let Ω be a set of [training] data $\{(\boldsymbol{x}_n, y_n), n = 1, \ldots, N\}$.
- $\varphi(\boldsymbol{x}, \Omega)$ is a [weak] predictor for y.
- $\{\Omega_k\}$ is a sequence of learning sets of N independent observations drawn from the same underlying distribution as Ω.
- The goal is to obtain a better prediction using this sequence $\{\varphi(\boldsymbol{x}, \Omega_k)\}$ than a single predictor $\varphi(\boldsymbol{x}, \Omega)$.
- For y numerical, we replace the single $\varphi(\boldsymbol{x}, \Omega)$ by the average of $\varphi(\boldsymbol{x}, \Omega_k)$, i.e.

$$\varphi_A(\boldsymbol{x}) = E_\Omega[\varphi(\boldsymbol{x}, \Omega_k)]$$

- For y a class $j \in \{1, \ldots, J\}$, we aggregate by voting. Let $N_j = \#\{k; \varphi(\boldsymbol{x}, \Omega_k) = j\}$, take

$$\varphi_A(\boldsymbol{x}) = \arg\max_j N_j$$

Bagging: formally II

BUT

- We only have one Ω, not several $\{\Omega_k\}$.
- Solution: imitate these $\{\Omega_k\}$ sets by taking repeated bootstrap samples $\{\Omega^B\}$ from Ω.
- $\{\Omega^B\}$ are replicate datasets drawn from the bootstrap distribution approximating the distibution underlying Ω (Efron and Thibshirani, 93).
- The final output is

$$\varphi_B(\boldsymbol{x}) = \text{aggregation}\left(\varphi(\boldsymbol{x}, \Omega^B)\right)$$

Why it works

- Let each pair $(\boldsymbol{x}, y)$ from Ω be independently drawn from the probability distribution P. The aggregated predictor is $\varphi_A(\boldsymbol{x}, P) = E_\Omega[\varphi(\boldsymbol{x}, \Omega)]$.
- Take $\boldsymbol{X}, Y$ random variables drawn from P and idependent of Ω. The average prediction error for $\varphi(\boldsymbol{x}, \Omega)$ is

$$e = E_{\boldsymbol{X},Y}[E_\Omega[(Y - \varphi(\boldsymbol{X}, \Omega))^2]]$$

- The prediction error for φ_A is

$$e_A = E_{\boldsymbol{X},Y}[(Y - \varphi_A(\boldsymbol{X}, \Omega))^2]$$

- Using inequality $(EZ)^2 \leq EZ^2$ gives

$$e = E_{\boldsymbol{X},Y}[Y^2] - 2E_{\boldsymbol{X},Y}[Y \cdot \varphi_A] + E_{\boldsymbol{X},Y}[E_\Omega[\varphi^2(\boldsymbol{X}, \Omega)]] \quad (1)$$

$$\geq E_{\boldsymbol{X},Y}[Y^2] - 2E_{\boldsymbol{X},Y}[Y \cdot \varphi_A] + E_{\boldsymbol{X},Y}[\varphi_A^2] = E_{\boldsymbol{X},Y}[(Y - \varphi_A)^2] = e_A \quad (2)$$

- This means φ_A has lower MSE error than φ. How much lower depends on how unequal these are

$$(E_\Omega[\varphi(\boldsymbol{x}, \Omega)])^2 \leq E_\Omega[\varphi^2(\boldsymbol{x}, \Omega)]$$

  which depends on how much $\varphi(\boldsymbol{x}, \Omega)$ changes with each replicate Ω.
- The more distinct $\varphi(\boldsymbol{x}, \Omega)$ are, the more improvement aggregation may produce.

Bagging: summing up

In general

- *Unstable predictors (i.e. NNs, trees)* better than *stable predictors (i.e. kNN)*[7].

In regression

- If replicates are very different, aggregation φ_B can improve results over $\varphi(\boldsymbol{x}, \Omega)$. Otherwise, φ_B may not improve.

In classification

- If a predictor is good for most inputs $\boldsymbol{x}$, aggregation can transform it into a nearly optimal classifier.
- But poor predictors can be transformed into worse ones!

[7] Breiman Tech. Report 421, 1994

Bagging pseudocode

Train: do $t = 1, \ldots, T$

1. Take bootstrapped replica Ω^k.
2. Train weak learner φ_k with Ω^k. (`DecisionTreeClassifier`, `DecisionTreeRegressor`)
3. Add φ_k to ensemble.

Test:

1. Predict new sample $\boldsymbol{x}$ on all ensemble learners, then
   - For classification: use majority vote to get the class
   - For regression: use the mean to get the function value

Bagging: scikit-learn implementation

- `BaggingClassifier` and `BaggingRegressor`
- Implements several methods:
  - Bagging: samples drawn with replacement (`bootstrap`, default mode).
  - Pasting: subsets drawn for random subsets of samples (`max_samples`).
  - Random subspaces: subsets drawn as random subsets of the features (`bootstrap_features`).
  - Random patches: pasting + random subspaces.

Exercice: Bagging

Implement and use implemented versions

1. Try with: `make_moons`, `make_circles`, `load_iris`
2. Try with real functions: `sinc` with added noise, `load_boston`, `load_diabetes`
3. Test other base classifier / regressors
4. Bonus: try to parallelize it! (`https://joblib.readthedocs.io/`)
5. Compare results with our implementation
6. Compare results of different algorithms

Random forests I

Like bagging, but ...

- Using always trees as base learners (thus their name).
- Bootstrap is still used to build the training sets.
- A small number of random features are selected when dividing **each node** of the trees. *In each split we select among all available features*.
- The goal is to decrease the correlation between the trees in forest.
- The variance is reduced when averaging the trees by the factor of the number of trees in the forest.

Random forests II

Advantages

- As accurate as AdaBoost or even better.
- Relatively robust to noise and outliers.
- Faster than bagging or boosting.
- Gives internal estimates of generalization error, strength, correlation and variable importance.

Random forests III

Estimation of the generalization error

- Let Ω the training set, $\{\Omega_k\}$ bootstrap training sets, and $\varphi(\boldsymbol{x}, \Omega_k)$ predictors.
- By using bootstrap, each tree is constructed using about 2/3 of the training data, leaving 1/3 for validation.
- When training each predictor φ_k, test each pair $y, \boldsymbol{x}$ not in Ω_k (called *out-of-bag* samples), and get a prediction.
- For each $y, \boldsymbol{x}$ in the training set, aggregate only over those predictors for which Ω_k does not contain $y, \boldsymbol{x}$. This is the *oob predictor*.
- The *oob estimate* of the generalization error is the error rate of the oob predictor on the training set.

Random forests IV

Variable importance

- Based on *permutation importance*.
- After each tree is constructed, the values of the m-th variable in the oob samples are randomly permuted and the oob data run down in that tree. The classification obtained for each $\boldsymbol{x}_n$ oob is saved.
- This is repeated for each variable $m = 1, \ldots, M$. The error results for each $\boldsymbol{x}_n$ and m variable are obtained.
- These errors are compared with the error committed with the variables intact. The higher the error, the higher the importance of the m-th variable.

RF: scikit-learn implementation

Implemented methods

- `RandomForestClassifier`
- `ExtraTreesClassifier`: Discriminative thresholds are drawn at random when growing the trees, this reduces the variance and slightly increases the bias.

Important RF hyper-parameters

- max_features: number of sampled features present at every split. This parameter is critical.
- min_samples_leaf: deep of the trees. If large increases the bias but diminishes the variance.
- n_estimators: number of weak estimators to use. In general, the larger the better (more computational cost).

Exercise: RF

Implement and sklearn

1. Implement RF.
2. Compare different hyperparameters.
3. Try with different functions: `sinc` with added noise, `make_moons`, `make_circles`, `load_iris`, `load_boston`, `load_diabetes`.
4. Use scikit-learn RF implementations: Compare between RF and ExtraTress algorithms.
5. Get OOB error estimations and feature importance.

Boosting

Robert Schapire proved in 1990 that a *weak learner* (WL) can be turned into a *strong learner* (SL).[8]

- Boosting algorithms iteratively train WLs w.r.t. a distribution and add them to build a final SL. Each time a new WL is added:
  1. The new WL is weighted taking into account its accuracy.
  2. The training data are re-weighted so wrong/right predicted samples gain/loose weight.
- The last step has the effect to force future learners to focus more on difficult samples.

Typical boosting algorithms:

- AdaBoost (and variants)
- Gradient boosting

[8] *The Strength of Weak Learnability*. Machine Learning, 5(2): 192–227

AdaBoost

Short for Adaptive Boosting, formulated by Yoav Freund and Robert Schapire in 1997[9].
A boost classifier has the form

$$F_T(\boldsymbol{x}) = \sum_{t=1}^{T} \alpha_t h_t(\boldsymbol{x})$$

where h_t are weak learners.
The goal is select at each iteration the weak learner that minimizes the training error

$$E_t = \sum_i \mathcal{E}[F_{t-1}(\boldsymbol{x}_i) + \alpha_t h(\boldsymbol{x}_i)]$$

where $\mathcal{E}[\cdot]$ is an error function.

[9] *A decision-theoretic generalization of on-line learning and an application to boosting*, Journal of Computer and Systems Sciences, 55(1), 119–139, 1997.

AdaBoost

We have a dataset $\{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_N, y_N)\}$, $y_i \in \{-1, 1\}$, and a set of weak classifiers $\{h_1, \ldots, h_L\}$ with outputs $h_j(\boldsymbol{x}_i) \in \{-1, 1\}$. After the $t-1$ iteration the classifier has the form

$$C_{t-1}(\boldsymbol{x}_i) = \sum_{j=1}^{t-1} \alpha_j h_j(\boldsymbol{x}_i)$$

At the t iteration the goal is to improve the classifier by adding one of the weak classifiers

$$C_t(\boldsymbol{x}_i) = C_{t-1}(\boldsymbol{x}_i) + \alpha_t h_t(\boldsymbol{x}_i)$$

We have to select both the classifier h_t and the weight α_t.

Selecting the weak classifier h_t

The error $\mathcal{E}$ is defined as the sum of the exponential loss of each point

$$\mathcal{E} = \sum_{i=1}^{N} e^{-y_i C_t(\boldsymbol{x}_i)} = \sum_{i=1}^{N} w_i^t e^{-y_i \alpha_t h_t(\boldsymbol{x}_i)}$$

where $w_i^t = e^{-y_i C_{t-1}(\boldsymbol{x}_i)}$, and $w_i^1 = 1/N$.
w_i^t will be the weight assigned to the i-th training sample at iteration t.
This way, each training sample is weighted according the exponential loss obtained by the weak classifier prediction.

Rewriting between correct and incorrect classified points:

$$\mathcal{E} = \sum_{y_i = h_t(\boldsymbol{x}_i)} w_i^t e^{-\alpha_t} + \sum_{y_i \neq h_t(\boldsymbol{x}_i)} w_i^t e^{\alpha_t} = \sum_{i=1}^{N} w_i^t e^{-\alpha_t} + \sum_{y_i \neq h_t(\boldsymbol{x}_i)} w_i^t (e^{\alpha_t} - e^{-\alpha_t})$$

The only part that depends on h_t is $\sum_{y_i \neq h_t(\boldsymbol{x}_i)}^{N} w_i^t$. Therefore, $\mathcal{E}$ is minimal when we choose the weak classifier with the lowest total cost, $h_t = \arg\min_{h_m} \sum_{y_i \neq h_m} w_i^m$.

Selecting the weight α_t

To obtain the optimal α_t we derive $\mathcal{E}$ and equal to zero.

$$\frac{\partial \mathcal{E}}{\partial \alpha_t} = \alpha_t \left(\sum_{y_i \neq h_t(\boldsymbol{x}_i)} w_i^t e^{\alpha_t} - \sum_{y_i = h_t(\boldsymbol{x}_i)} w_i^t e^{-\alpha_t} \right) = 0$$

Therefore

$$\alpha_t = \frac{1}{2} \ln \left(\frac{\sum_{y_i = h_t(\boldsymbol{x}_i)} w_i^t}{\sum_{y_i \neq h_t(\boldsymbol{x}_i)} w_i^t} \right)$$

Defining the error rate of the classifier as $\varepsilon_t = \frac{\sum_{y_i \neq h_t(\boldsymbol{x}_i)} w_i^t}{\sum_{i=1}^N w_i^t}$

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

which is half the negative logit function.

AdaBoost: about how sample weights are updated

Sample weights updating

$$w_i^{t+1} = w_i^t e^{-\alpha_t y_i h_t(\boldsymbol{x}_i)}$$

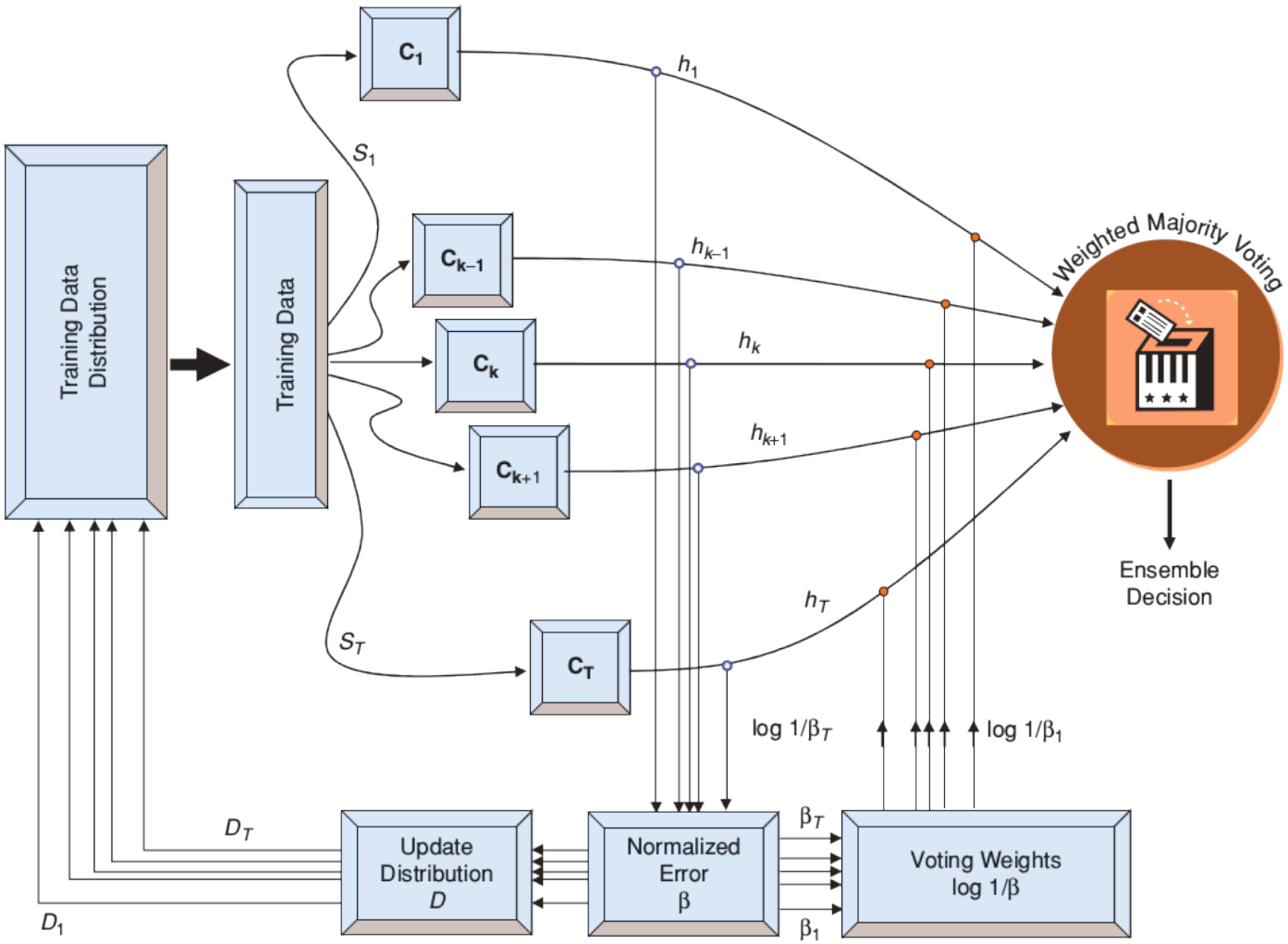
$$w_i^{t+1} = \frac{w_i^{t+1}}{\sum_i w_i^{t+1}}$$

- Classifiers that do right cause sample weights to *decrease*
- Those that do wrong cause sample weights to *increase*

Consequences:

1. Training samples harder to classify get more importance
2. Each sample weight is an upper bound of the error of a given sample

AdaBoost: structure



AdaBoost algorithm

For each iteration t

1. Train a classifier h_t that minimizes the total weighted error $\sum_{y_i \neq h_m(\boldsymbol{x}_i)} w_i^m$.
2. Set the error rate ε_t.
3. Set the weights α_t and w_t.
4. Obtain the next classifier as $C_t = C_{t-1} + \alpha_t h_t$.

AdaBoost algorithm (M1 variant)

Initialize weights $w_i^1 = 1/N$

For each iteration t

1. Weight training set according to w_i^t and train weak learner(s).
2. Calculate the error rate $\varepsilon_t = \sum_{y_i \neq h_t(\boldsymbol{x}_i)} w_i^t$.
3. Stop if $\varepsilon_t \geq 1/2$
4. Set the normalized error $\beta_t = \frac{\varepsilon_t}{1-\varepsilon_t}$
5. Set the weight $\alpha_t = \ln \frac{1}{\beta_t}$
6. Update $w_i^{t+1} = w_i^t \times \begin{cases} \beta_t & \text{if } h_t(\boldsymbol{x}_i) = y_i \\ 1 & \text{otherwise} \end{cases}$
7. Re-normalize weights $w^{t+1} = w^{t+1} / \sum_{i=1}^N w^{t+1}$.

Testing

Choose $\arg\max_{y_j} V_j = \sum_{t; h_t(\boldsymbol{x}) = y_j} \alpha_t \qquad j = 1, \ldots, T$.

Exercise: AdaBoost

Implement

1. Implement and compare original AdaBoost and the M1 variant.
2. Try it with: `make_circles`, `make_moons`, `make_classification`.
3. Save the history of weights (w_i^t) and errors (ε_t, β_t) at different iterations and represent them.
4. Try different numbers of weak learners in the ensemble.

AdaBoost scikit-learn implementation

- `AdaBoostClassifier` and `AdaBoostRegressor`

Important hyper-parameters:

- n_estimators: Large values increase the variance, low values increase the bias.
- max_depth: Increases the variance and complexity.
- subsample: Can reduce the variance.
- learning_rate: Smaller values improve optimization, but requires more estimators to converge.
- min_samples_leaf: Reduce the variance due to noisy data. Avoids overfitting.

AdaBoost for regression

- The form of the model is the same: $F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$
- At each iteration we want to add a new predictor that minimizes the training error.
- Consider $F_1(x)$. We obtain $F_2(x) = F_1(x) + \alpha_2 h_2(x)$, where $h_2(x)$ is a new predictor (usually a tree).
- For all the training samples, we want a new predictor that fixes the errors commited by the previous ones, that is

$$y_1 = F_1(x_1) + \alpha_2 h_2(x_1) \rightarrow \alpha_2 h_2(x_1) = y_1 - F_1(x_1)$$
$$\vdots$$
$$y_n = F_1(x_n) + \alpha_2 h_2(x_n) \rightarrow \alpha_2 h_2(x_n) = y_n - F_1(x_n)$$

- We achieve or goal by training $h_t(x)$ using the pairs $\{x_i; y_i - F_{t-1}(x_i)\}$ as training set.

AdaBoost Regression training I

Cost function $\mathcal{L} = \frac{1}{2}\|y - F_t(\boldsymbol{x})\|^2$, or

$$\mathcal{L} = \frac{1}{2}\sum_i (y_i - F_t(\boldsymbol{x}_i))^2 = \frac{1}{2}\sum_i (y_i - F_{t-1}(\boldsymbol{x}_i) - \alpha_t h_t(\boldsymbol{x}_i))^2$$

Derivative:

$$\frac{\partial \mathcal{L}}{\partial \alpha_t} = -\sum_i \left(h_t(\boldsymbol{x}_i) \cdot (y_i - F_{t-1}(\boldsymbol{x}_i) - \alpha_t h_t(\boldsymbol{x}_i))\right)$$

Equaling to zero we obtain:

$$\alpha_t = \frac{\sum_i \left(h_t(\boldsymbol{x}_i) \cdot (y_i - F_{t-1}(\boldsymbol{x}_i))\right)}{\sum_i h_t(\boldsymbol{x}_i)^2}$$

The weights are updated using the loss function (MSE) on the samples:

$$w_i = \frac{1}{2}\left(y_i - F_{t-1}(\boldsymbol{x}_i) - \alpha_t h_t(\boldsymbol{x}_i)\right)^2$$

AdaBoost Regression training II

A more general procedure is this. Given a loss function $\mathcal{L}(y, F_t(\boldsymbol{x}_i))$ obtain the optimal α_t as

$$\alpha_t = \arg\min_{\alpha_t}\{\mathcal{L}(y, F_t(\boldsymbol{x}_i) + \alpha_t h_t(\boldsymbol{x}_i))\}$$

In the case of the least-squares loss function this is

$$\arg\min_{\alpha_t}(y_i - F_{t-1}(\boldsymbol{x}_i) - \alpha_t h_t(\boldsymbol{x}_i))^2$$

For the least absolute value loss is

$$\arg\min_{\alpha_t}|y_i - F_{t-1}(\boldsymbol{x}_i) - \alpha_t h_t(\boldsymbol{x}_i)|$$

These can be solved using the *line search algorithm*[10].

[10] https://en.wikipedia.org/wiki/Line_search

Gradient boosting I

- Instead of training the models using on the residuals, we rely on gradient descent.
- Models (or parameters) are updated following the direction of the negative gradient of a defined loss function w.r.t. parameters.

$$\boldsymbol{\theta}_i = \boldsymbol{\theta}_i - \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}_i}$$

- Lets define a quadratic loss function as $\mathcal{L} = \frac{1}{2}\sum_i (F(x_i) - y_i)^2$.
- Thus $g(x_i) \equiv \frac{\partial \mathcal{L}}{\partial F(x_i)} = F(x_i) - y_i$.
- The negative gradient is $-g(x_i) = y_i - F(x_i)$, the residuals!
- Now we train new models $h_t(x)$ using pairs $\{x_i; -g(x_i)\}$ as training sets.

Greedy Function Approximation: A Gradient Boosting Machine. Friedman, 2001.

Gradient boosting II

- When using the squared loss we obtain the same, but now we have a general procedure, and we can use different loss functions.
- The squared loss function is quite sensitive to outliers.
- Other possible loss functions
  - L_1 norm (absolute error) loss function:

$$\mathcal{L} = \sum_i |y_i - F(x_i)|$$

$$-g(x_i) = \text{sign}(y_i - F(x_i))$$

  - Hubber loss function:

$$\mathcal{L} = \begin{cases} \frac{1}{2}(F(x_i) - y_i)^2 & \text{if } |y_i - F(x_i)| < \varepsilon \\ \varepsilon(|y_i - F(x_i)| - \varepsilon/2) & \text{if } |y_i - F(x_i)| \geq \varepsilon \end{cases}$$

$$-g(x_i) = \begin{cases} y_i - F(x_i) & \text{if } |y_i - F(x_i)| < \varepsilon \\ \varepsilon \cdot \text{sign}(y_i - F(x_i)) & \text{if } |y_i - F(x_i)| \geq \varepsilon \end{cases}$$

Exercise: AdaBoost/Gradient boost

Sklearn

1. Play with the scikit-learn `AdaBoostRegressor` and `GradientBoostingRegressor`.
2. Try different loss functions.
3. Once trained, get the following variables and analyze them:
   1. Estimator weights (`estimator_weights_`).
   2. Estimator errors (`estimator_errors_`).
   3. Feature/variable importance (`feature_importances_`).
4. Is it possible to use any base regressor? Explain.

XGBoost

- Introduced by Chen et al. 2016 (`https://xgboost.readthedocs.io/en/latest/`)
- Gradient boosting plus
  1. Added additional regularization term.
  2. Shrinkage: scales new added weights by a factor η after each step of tree boosting.
  3. Column subsampling: random feature selection (like in RFs).
  4. Looks for the best splits based on histograms and quantile approaches.
  5. *Sparsity-aware split finding*: When a feature is missing a default direction is used to classify an instance.
  6. Eacth tree is grown in parallel.

Gradient boosting libraries to consider

- Scikit-learn: `https://scikit-learn.org/stable/modules/ensemble.html`
  - GradientBoostingClassifier / GradientBoostingRegressor: for no too big datasets.
  - HistGradientBoosting...: Histogram based. Much faster with tens of thousands of samples. Handles missing and categorical data.
- XGBoost `https://xgboost.readthedocs.io/en/latest/`
- LightGBM: `https://lightgbm.readthedocs.io/en/stable/` Maybe the best library in terms of accuracy, speed and capacity to work with many samples.
- CatBoost: `https://catboost.ai/` Specially appropiate for categorical or text data. Very fast also, works with GPU.

Introduction

Basic
Combining labels
Combining outputs
Model Based

Bagging

Random Forests

Boosting
AdaBoost
Gradient boosting

Machine Learning: Ensembles

Machine Learning I - MCD

J. Muñoz and V. Laparra

ISP - ETSE - Universitat de València

VLC - 2024